

12

النوع Tuple : فهم الصفوف

يبدو نوع البيانات tuple (صف) في بايثون كما يلي:

```
coral = ('blue coral', 'staghorn coral', 'pillar coral',
        'elkhorn coral')
```

النوع tuple (صف وتُجمع إلى صفوف) هي بنية بيانات تُمثّل سلسلة مرتبة من العناصر غير القابلة للتبديل، وبالتالي لا يمكن تعديل القيم الموجودة فيها. يستعمل نوع البيانات tuple لتجميع البيانات، فكل عنصر أو قيمة داخل الصف تُشكّل جزءاً منه. توضع القيم داخل الصف بين قوسين () ويُفصل بينها بفاصلة أجنبية , وتبدو القيم الفارغة كما يلي () coral = , لكن إذا احتوى الصف على قيم -حتى لو كانت قيمةً واحدةً فقط- فيجب وضع فاصلة فيه مثل coral = ('blue coral',). إذا استخدمنا الدالة print() على النوع tuple، فسنحصل على الناتج الآتي الذي يُبيّن أنّ القيمة الناتجة ستوضع بين قوسين:

```
print(coral)
('blue coral', 'staghorn coral', 'pillar coral', 'elkhorn coral')
```

عند التفكير بالنوع tuple وغيره من بنى البيانات التي تُعدّ من أنواع «التجميعات» (collections)، فمن المفيد أن تضع ببالك مختلف التجميعات الموجودة في حاسوبك: تشكيلة الملفات الموجودة عندك، وقوائم التشغيل للموسيقى، والمفضلة الموجودة في متصفحك، ورسائل بريدك الإلكتروني، ومجموعة مقاطع الفيديو التي تستطيع الوصول إليها من التلفاز، والكثير. نوع tuple (صف) شبيه بالنوع list (قائمة)، لكن القيم الموجودة فيه لا يمكن تعديلها، وبسبب ذلك، فأنت تخبر الآخرين أنّك لا تريد إجراء أيّة تعديلات على هذه السلسلة من القيم عندما تستعمل النوع tuple في شيفرتك. إضافةً إلى ما سبق، ولعدم القدرة على تعديل القيم،

فسيكون أداء برنامجك أفضل، حيث ستنفذ الشيفرة بشكل أسرع إذا استعملت الصفوف بدلاً من القوائم (lists).

1. فهرسة الصفوف

يمكن الوصول إلى كل عنصر من عناصر الصف بمفرده لأنّه سلسلة مرتبة من العناصر، وذلك عبر الفهرسة. وكل عنصر يرتبط برقم فهرس، الذي هو عدد صحيح يبدأ من الفهرس 0. ستبدو الفهارس من مثال coral السابق والقيم المرتبطة بها كالآتي:

elkhorn coral	pillar coral	staghorn coral	blue coral
3	2	1	0

العنصر الأول الذي يُمثّل السلسلة النصية 'blue coral' فهرسه 0، وتنتهي القائمة بالفهرس رقم 3 المرتبط بالقيمة 'elkhorn coral'. ولأنّ كل عنصر من عناصر الصف له رقم فهرس مرتبط به، فسنتمكن من الوصول إلى عناصره فرادى. يمكننا الآن الوصول إلى عنصر معيّن في الصف عبر استخدام رقم الفهرس المرتبط به.

```
print(coral[2])
pillar coral
```

تتراوح قيم الفهارس في المتغير coral من 0 إلى 3 كما هو ظاهر في الجدول السابق، لذا يمكننا استدعاء العناصر الموجودة فيه فرادى كما يلي:

```
coral[0]
coral[1]
coral[2]
```

```
coral[3]
```

إذا حاولنا استدعاء المتغير coral مع رقم فهرس أكبر من 3، فستظهر رسالة خطأ تشير إلى أنَّ الفهرس خارج المجال:

```
print(coral[22])
# IndexError: tuple index out of range
```

إضافةً إلى أرقام الفهارس الموجبة، يمكننا أيضًا الوصول إلى الفهارس باستخدام رقم فهرس سالب، وذلك بالعد بدءًا من نهاية قائمة العناصر وسيرتبط آخر عنصر بالفهرس -1، وهذا مفيد جدًا إذا كان لديك متغير من النوع tuple وكان يحتوي عناصر كثيرة وأردت الوصول إلى أحد عناصره انطلاقًا من النهاية. ففي مثالنا السابق عن coral، إذا أردنا استخدام الفهارس السالبة فالناتج كالاتي:

elkhorn coral	pillar coral	staghorn coral	blue coral
-1	-2	-3	-4

إذا أردنا طباعة العنصر 'blue coral' باستخدام الفهارس السالبة، فستبدو التعليمة كما يلي:

```
print(coral[-4])
blue coral
```

يمكننا إضافة العناصر النصية الموجودة في الصف إلى السلاسل النصية الأخرى باستخدام

العامل +:

```
print('This reef is made up of ' + coral[1])
# This reef is made up of staghorn coral
```

استطعنا في المثال السابق إضافة عنصر موجود في الفهرس 1 مع السلسلة النصية 'This reef is made up of '، ويمكننا أيضًا استخدام العامل + لإضافة بنيتي صف معًا.

2. تقطيع قيم صف

يمكننا استخدام الفهارس للوصول إلى عدّة عناصر من صف، أما التقطيع فيسمح لنا بالوصول إلى عدّة قيم عبر إنشاء مجال من أرقام الفهارس المفصولة بنقطتين رأسيّتين [x:y]. لنقل أننا نريد عرض العناصر الموجودة في وسط المتغير coral، يمكننا فعل ذلك بإنشاء قطعة جديدة:

```
print(coral[1:3])
('staghorn coral', 'pillar coral')
```

عند إنشاء قطعة جديدة -كما في المثال السابق- فيمثّل أوّل رقم مكان بدأ القطعة (متضمنةً هذا الفهرس)، ورقم الفهرس الثاني هو مكان نهاية القطعة (دون تضمين هذا الفهرس بالقطعة)، وهذا هو السبب وراء عرض المثال السابق للقيم المرتبطة بالعناصر الموجودة في الفهرسين 1 و 2. إذا أردت تضمين إحدى نهايتي القائمة، فيمكنك حذف أحد الأرقام في التعبير tuple[x:y]، فمثلاً، لنقل أننا نريد عرض أوّل ثلاثة عناصر من coral، والتي هي 'blue coral' و 'staghorn coral' و 'pillar coral'، فيمكننا فعل ذلك كالآتي:

```
print(coral[:3])
('blue coral', 'staghorn coral', 'pillar coral')
```

المثال السابق عرض العناصر من بداية القائمة وتوقف قبل العنصر ذي الفهرس 3. لتضمين

كل العناصر الموجودة في نهاية الصف، فيمكننا عكس التعبير السابق:

```
print(coral[1:])
# ('staghorn coral', 'pillar coral', 'elkhorn coral')
```

يمكننا استخدام الفهارس السالبة أيضًا عند التقطيع، كما فعلنا مع أرقام الفهارس الموجبة:

```
print(coral[-3:-1])
print(coral[-2:])
# ('staghorn coral', 'pillar coral')
# ('pillar coral', 'elkhorn coral')
```

هناك معامل إضافي يمكننا استعماله ويسمى «الخطوة»، ويشير إلى عدد العناصر التي يجب تجاوزها بعد الحصول على أول عنصر من القائمة. حذفنا في جميع أمثلتنا السابقة معامل الخطوة، إذ القيمة الافتراضية له في بايثون هي 1، لذا سنحصل على جميع العناصر الموجودة بين الفهرسين المذكورين. شكل هذا التعبير العام هو `tuple[x:y:z]`، إذ يُشير المعامل `z` إلى الخطوة. لنُنشئ قائمة أكبر، ثم نقسمها، ونعطيها القيمة 2 للخطوة:

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12)
print(numbers[1:11:2])
# (1, 3, 5, 7, 9)
```

ستطبع التعليمة `numbers[1:11:2]` القيم الموجودة بين رقمين الفهرسين 1 (بما في ذلك العنصر المرتبط بالفهرس 1) و 11 (دون تضمين ذلك العنصر)، ومن ثم ستخبر قيمة الخطوة 2 البرنامج أن يتخطى عنصرًا بين كل عنصرين. يمكننا حذف أول معاملين واستخدام معامل الخطوة بمفرده بتعبير برمجي من الشكل `tuple[:,z]`:

```
print(numbers[:3])
# (0, 3, 6, 9, 12)
```

طبعنا في المثال السابق عناصر numbers بعد ضبط قيمة الخطوة إلى 3، وبالتالي سيتم تخطي عنصرين.

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
```

تقطيع الصفوف باستخدام أرقام الفهارس الموجبة والسالبة واستعمال معامل الخطوة يسمح لنا بالتحكم بالناتج الذي نريد عرضه.

3. إضافة بنى صف إلى بعضها

يمكن أن نُضيف بنى صف إلى بعضها أو أن «نضربها» (multiply)، تتم عملية الإضافة باستخدام المعامل +، أما عملية الضرب فباستخدام المعامل *. يمكن أن يُستخدَم المعامل + لإضافة بنيتي صف أو أكثر إلى بعضها بعضًا. يمكننا إسناد القيم الموجودة في بنيتي صف إلى بنية جديدة:

```
coral = ('blue coral', 'staghorn coral', 'pillar coral',
        'elkhorn coral')
kelp = ('wakame', 'alaria', 'deep-sea tangle', 'macrocystis')

coral_kelp = (coral + kelp)

print(coral_kelp)
```

الناتج:

```
('blue coral', 'staghorn coral', 'pillar coral', 'elkhorn
coral', 'wakame', 'alaria', 'deep-sea tangle', 'macrocystis')
```

وصحيحٌ أنَّ المعامل + يمكنه إضافة بنى صف إلى بعضها، لكن يمكن أن يستعمل لإنشاء بنية صف جديدة ناتجة عن جمع بنى أخرى، لكن لا يمكنه تعديل بنية صف موجودة مسبقًا. أما العامل * فيمكن استخدامه لضرب بنى صف، فربما تريد إنشاء نسخ من الملفات الموجودة في أحد المجلدات إلى الخادوم أو مشاركة قائمة بالمقطوعات الموسيقية التي تحبها مع أصدقائك، ففي هذه الحالات سترغب بمضاعفة مجموعات من البيانات (أو «ضربها»). لنضرب البنية coral بالرقم 2 والبنية kelp بالرقم 3، ثم نسندھا إلى بنى صف جديدة:

```
multiplied_coral = coral * 2
multiplied_kelp = kelp * 3

print(multiplied_coral)
print(multiplied_kelp)
```

الناتج:

```
('blue coral', 'staghorn coral', 'pillar coral', 'elkhorn
coral', 'blue coral', 'staghorn coral', 'pillar coral',
'elkhorn coral')
('wakame', 'alaria', 'deep-sea tangle', 'macrocystis',
'wakame', 'alaria', 'deep-sea tangle', 'macrocystis', 'wakame',
'alaria', 'deep-sea tangle', 'macrocystis')
```

يمكننا باستخدام العامل * أن نُكرِّر (أو نُضاعِف) بنى صف بأي عدد من المرات نشاء، مما سيُنشئ بنى صف جديدة اعتمادًا على محتوى البنى الأصلية. خلاصة ما سبق هي أنَّ بنى الصف يمكن إضافتها إلى بعضها أو ضربها لتشكيل بنى صف جديدة عبر استخدام العاملين + و *.

4. دوال التعامل مع الصفوف

هناك دوال مُضمَّنة في لغة بايثون للتعامل مع بنى النوع tuple، لننظر إلى بعضها.

1. len()

وكما في السلاسل النصية والقوائم، يمكننا حساب طول (أو عدد عناصر) بنية صف باستخدام الدالة len(). إذ تُمرَّر إليها بنية صف (معامل)، كما يلي:

```
len(coral)
```

هذه الدالة مفيدة إذا أردنا أن نَضمَّن أنَّ لبنية صف عدد عناصر معيَّن، فمثلاً يمكننا الاستفادة من ذلك بموازنة بنيتين مع بعضهما. إذا أردنا طباعة عدد عناصر kelp و numbers، فسيظهر الناتج الآتي:

```
print(len(kelp))
print(len(numbers))
```

الناتج:

```
4
13
```

يشير الناتج أعلاه إلى أنَّ لبنية kelp أربعة عناصر:

```
kelp = ('wakame', 'alaria', 'deep-sea tangle', 'macrocystis')
```

أما البنية numbers فتتملك ثلاثة عشر عنصراً:

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12)
```

وصحيح أنَّ هذه الأمثلة عناصرها قليلة نسبيًا، إلا أنَّ الدالة `len()` تستطيع أن تخبرنا بعدد عناصر بني tuple الكبيرة.

ب. الدالتان `min()` و `max()`

عندما نتعامل مع بني صف مكوَّنة من عناصر رقمية (بما فيها الأعداد الصحيحة والأرقام ذات الفاصلة العشرية)، فيمكننا استخدام الدالتين `min()` و `max()` للعثور على أكبر وأصغر قيمة موجودة في بنية صف معيَّنة. تسمح لنا هاتان الدالتان باستخراج معلومات تخص البيانات القابلة للإحصاء، مثل نتائج الامتحانات أو درجات الحرارة أو أسعار المنتجات... إلخ. لننظر إلى بنية صف مكونة من أعداد عشرية:

```
more_numbers = (11.13, 34.87, 95.59, 82.49, 42.73, 11.12,
95.57)
```

للحصول على القيمة العظمى من بين القيم الآتية فعلينا تمرير بنية صف إلى الدالة `max()`

كما في `max(more_numbers)`، وسنستخدم الدالة `print()` لعرض الناتج:

```
print(max(more_numbers))
# 95.59
```

أعادت الدالة `max()` أعلى قيمة في بنية `more_numbers`. وبشكلٍ شبيهٍ بما سبق نستخدم

الدالة `min()`:

```
print(min(more_numbers))
# 11.12
```

أعيدَ هنا أصغر رقم عشري موجودة في البنية. يمكن الاستفادة من الدالتين `max()`

و `min()` كثيرًا للتعامل مع بني tuple التي تحتوي الكثير من القيم.

5. كيف تختلف بنى الصفوف عن القوائم

الفرق الرئيسي بين النوع tuple والنوع list هو عدم القدرة على تعديل العناصر، وهذا يعني أننا لا نستطيع إضافة أو حذف أو استبدال العناصر داخل بنية tuple. لكن يمكننا إضافة بنيتي tuple أو أكثر إلى بعضها بعضًا لتشكيل بنية جديدة كما رأينا في أحد الأقسام السابقة. لتكن لدينا البنية coral الآتية:

```
coral = ('blue coral', 'staghorn coral', 'pillar coral',
        'elkhorn coral')
```

لنقل أننا نريد استبدال العنصر 'blue coral' ووضع العنصر 'black coral' بدلاً منه. فلو حاولنا تغيير بنية صف بنفس الطريقة التي نُعدّل فيها القوائم بكتابة:

```
coral[0] = 'black coral'
```

فستظهر رسالة خطأ كالآتية:

```
TypeError: 'tuple' object does not support item assignment
```

وذلك بسبب عدم إمكانية تعديل بنى الصفوف. إذا أنشأنا بنية صف ثم قررنا أن ما نحتاج له هو بنية قائمة، فيمكننا تحويلها إلى قائمة list، وذلك بالدالة list():

```
list(coral)
```

أصبحت بنية coral قائمة الآن:

```
coral = ['blue coral', 'staghorn coral', 'pillar coral']
```

يمكننا أن نلاحظ أن بنية الصف tuple تحوَّلت إلى قائمة list لأنَّ الأقواس المحيطة بالقيم أصبحت مربعة الشكل.

وبشكلٍ شبيهٍ بما سبق، نستطيع تحويل القوائم من النوع list إلى tuple باستخدام

الدالة tuple().

6. خلاصة الفصل

نوع البيانات tuple (الصفوف) هو مجموعة من البيانات المتسلسلة التي لا يمكن تعديلها، ويوفّر تحسّينًا في أداء برامجك لأنه أسرع معالجةً من القوائم في بايثون. وعندما يراجع الآخرون شيفرتك فسيعلمون من استخدامك لبنى tuple أنك لا تريد تعديل هذه القيم. شرحنا في هذا الفصل الميزات الأساسية لبنى tuple بما في ذلك الفهارس وتقطيعها وتجميعها، وعرضنا بعض الدوال المُضمّنة المتوافرة لهذا النوع من البيانات.